



iEnhancer-BERT: A Novel Transfer Learning Architecture Based on DNA-Language Model for Identifying Enhancers and Their Strength

Hanyu Luo¹, Cheng Chen¹, Wenyu Shan¹, Pingjian Ding¹, and Lingyun Luo^{1,2}(✉)

¹ School of Computer Science, University of South China, Hengyang 421001, Hunan, China
luoly@usc.edu.cn

² Hunan Medical Big Data International Science and Technology Innovation Cooperation Base,
Hengyang 421001, China

Abstract. Enhancers are small segments of DNA that bind to proteins (transcription factors) and the transcription of a gene is strengthened after binding to the protein, thus playing an essential role in gene expression. Recently, machine learning-based methods have become a trend in identifying enhancers and their strength. In this study, we propose iEnhancer-BERT, a novel transfer learning method based on pre-trained DNA language model using the whole human genome. More specifically, iEnhancer-BERT consists of a BERT layer for feature extraction and a CNN layer for classification. We initialize our parameters of the BERT layer using a pre-trained DNA language model, and fine-tune it with transfer learning on the enhancer identification tasks. Unlike common fine-tuning strategies, we extract the output of all Transformer Encoder layers to form the feature vector. Experiments show that our method achieves state-of-the-art results in both enhancer identification tasks and strong enhancer identification tasks. The code and data are publicly available at <https://github.com/lhy0322/iEnhancer-BERT>.

Keywords: Enhancer identification · DNA sequence · BERT · Transfer learning · Convolutional neural network

1 Introduction

In genetics, enhancer is a DNA fragment of 50–1500 bp in the non-coding region of a gene, which is deemed to be the most critical typical regulatory element responsible for regulating the transcription of its target gene [1, 2]. Many diseases are associated with genetic variation in enhancers [3, 4]. However, identifying enhancers and their strength is a challenging task in biology. Many methods based on wet experiments have been proposed successively, such as ChIP-seq [5], histone modification [6, 7], DNase-seq [8] and ATAC-seq [9]. However, these experimental methods are very time-consuming and expensive.

On this account, many computational-based prediction methods have been established to rapidly identify enhancers in genes, including CSI-ANN [10], ChromaGenSVM [11], RFECS [12], EnhancerFinder [13], gkm-SVM [14] and BiRen [15]. iEnhancer-2L

[16] is the first model that can predict both enhancers and their strengths and has provided the first benchmark dataset. EnhancerPred [17], iEnhancer-CNN [18], iEnhancer-EL [19], iEnhancer-XG [20] and SENIES [21] build models on this benchmark dataset. Even though these methods show excellent performances, the accuracy of predictors needs further improvement.

In recent years, models of bidirectional encoder representations from Transformers (BERT) have attracted a lot of concern in the field of natural language processing [22]. BERT is a pre-trained model with masked language model (MLM) training on the English BookCorpus and Wikipedia datasets that can be fine-tuned by transfer learning in downstream tasks. Experimentally proven, BERT achieves state-of-the-art performance on a variety of downstream tasks. BERT-2D [23] is the first model to leverage the BERT architecture to extract sequence features in the enhancer identification tasks. BERT-2D treats a single nucleotide as a word fed into the BERT model and then use deep learning techniques to learn the features for classification. However, it has two drawbacks. Firstly, it used the natural language pre-trained model that do not contain any prior knowledge about biological sequences. Secondly, it treats a single nucleotide as a word input of the BERT model, thus can not capture the contextual information well. Ji et al. reported DNABERT [24], the first BERT-based DNA language model pre-trained on the whole human reference genome. It splits the gene sequence into 512 bp segments, which are then fed into the 12-layer Transformer architecture for MLM pre-training. With simple fine-tuning, it achieves excellent performance in many DNA-sequence-related downstream tasks even on small datasets.

In this study, we propose a novel transfer learning model based on DNABERT, termed iEnhancer-BERT, which not only overcomes two drawbacks of BERT-2D but also provides a novel fine-tuning architecture. It consists of a BERT layer and a CNN layer to accomplish the recognition task in two stages. The first stage is to identify enhancers and the second stage is to differentiate strong enhancers from weak enhancers. Experiments show that iEnhancer-BERT can achieve state-of-the-art performance on benchmark datasets. We believe that it can provide a new way of thinking for biological sequence modeling.

2 Materials and Methods

2.1 Benchmark Datasets

For this study, we retrieved the enhancer dataset from the iEnhancer-2L study collated by Liu et al. [16]. The dataset was built on 15 chromatin states of 9 cell types: GM12878, K562, HUVEC, HepG2, H1ES, HSMM, HMEC, NHEK, NHLF [25]. Liu et al. split the obtained enhancer sequences into several 200 bp fragments and removed sequences with greater than 20% similarity using the CD-HIT tool [26]. The benchmark dataset consists of 2968 samples, containing 1484 non-enhancers, 742 strong enhancers and 742 weak enhancers. In addition, the dataset provides an independent test dataset consisting of 400 samples, containing 200 non-enhancers, 100 strong enhancers and 100 weak enhancers. In summary, the dataset can be formulated as follows:

$$E = E^+ + E^- \quad (1)$$

$$E^+ = E_{strong}^+ + E_{weak}^+ \quad (2)$$

where E is the total number of all samples, E^+ is the number of enhancers, E^- is the number of non-enhancers, E_{strong}^+ is the number of strong enhancers, and E_{weak}^+ is the number of weak enhancers.

2.2 Methods

Figure 1 shows the architecture of iEnhancer-BERT consisting of two layers: BERT layer and CNN layer. First, we preprocess the input 200 bp sequence using the k-mer method, and then consider each k-mer token as a word input 12-layer Transformer Encoder to extract contextual information of the sequences. Next, the high-dimensional data combining all features of the 12-layer Transformer Encoder are fed into a convolutional neural network for further feature extraction. Finally, the flattened feature vectors are fed into a multi-layer feedforward neural network to complete the classification. Our models are described in detail following.

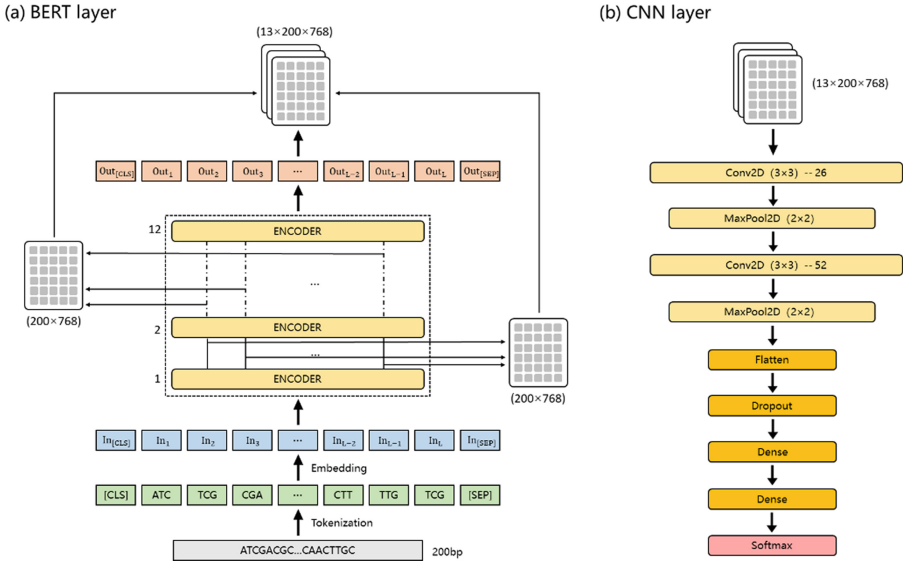


Fig. 1. The architecture of iEnhancer-BERT mainly composed of two layers: (a) BERT layer (b) CNN layer.

BERT Layer

The BERT layer consists of three modules: the sequence pre-processing module, the embedding module and the feature extraction module. In the sequence pre-processing module, every DNA sequence of length L needs to be cut into consecutive k-mer fragments. As a result, $L - k + 1$ fragments will be obtained. For example, given a sequence

“TCATGG”, the 3-mers of it is {TCA, CAT, ATG, TGG}. The k-mer division method is widely used in biological sequence pre-processing. Assuming that S_L^k is an input sequence of length L , the result of k-mer division is formulated as follows:

$$S_L^k = kmer(S_{L+k-1}) = \{s_1^k, s_2^k, s_3^k, \dots, s_{L-1}^k, s_L^k\} \quad (3)$$

where S_L^k indicates the output of k-mer division, s_i^k represents the substring of length k starting from the i -th position.

The k-mer sequence is tokenized before being fed into the embedding module. Two special tokens are inserted at the beginning ([CLS]) and the end ([SEP]) of the tokenization sequence separately, as follows:

$$T = Token(S_L^k) = [t_{[CLS]}, t_1, t_2, t_3, \dots, t_L, t_{[SEP]}] \quad (4)$$

In the formula, T indicates the output of tokenization. t_i denotes the token of the i -th substring with length k . Each token is fed into the embedding module and thus transformed into a vector. The embedding module consists of token embeddings and position embeddings, defined as follows:

$$E = Tok_Emb(T) + Pos_Emb(T) \quad (5)$$

where E indicates the output of the embedding module, $Tok_Emb(\cdot)$ and $Pos_Emb(\cdot)$ define the token embedding that can extract semantic information and the position embedding that can extract location information, respectively. The token embedding is to convert each token into a vector of fixed dimensions. In iEnhancer-BERT, each token is converted into a 768-dimensional vector representation. The position embedding allows the model to learn a vector representation at each position to encode information about the sequence order.

In the feature extraction module, we initialize the parameters of the BERT layer model using the pre-trained model provided by DNABERT. As shown in Fig. 1a, unlike DNABERT that only takes the header vector [CLS] of the last Transformer Encoder layer output as the representation of the whole sequence, we utilize all the token outputs of all of the Transformer Encoder layers. Considering the output of each layer of Transformer Encoder as one channel of the feature vector, the output of all layers (including the input of the first layer) are concatenated together to generate a 13-channel feature vector. Formally, the operation of feature extraction is defined as follow:

$$X = Concat(E_0, E_1, \dots, E_{12}) \quad (6)$$

$$E_i = \begin{cases} E & \text{if } i = 0 \\ ENCODER_i(E_{i-1}) & \text{if } i > 0 \end{cases} \quad (7)$$

where X indicates the output of the BERT layer, $Concat(\cdot)$ is the concatenation operation, and $ENCODER_i(\cdot)$ denotes the output of the i -th Transformer Encoder layer.

The Transformer Encoder layer consists of three modules, Multi-Head Attention, Add & Normalize and Feed Forward, with the internal details shown in Fig. 2. The Encoder obtains an optimized feature vector that focuses on the association of each part

of the sequence with the other parts by overlaying the attention mechanism and the feedforward neural network, as follows:

$$\begin{cases} H_i = E_{i-1} + MHA(E_{i-1}) \\ E_i = H_i + FFN(H_i) \end{cases} \quad i > 0 \quad (8)$$

where E_i indicates the output of the $ENCODER_i(\cdot)$, H_i is the intermediate vector of the i -th Encoder, $MHA(\cdot)$ define the multi-head self-attention module that captures information about the encoded representations in different subspaces and enhances the representational power of the model, and $FFN(\cdot)$ define a feed forward module consisting of two fully connected layers.

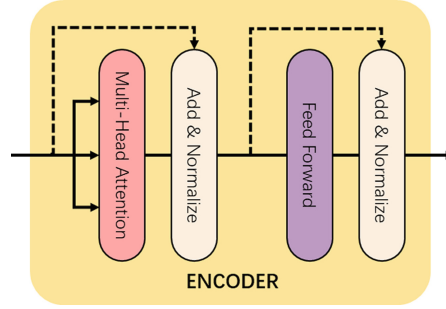


Fig. 2. The detail of Transformer Encoder layer.

CNN Layer

The CNN layer consists of two sub-modules: the convolutional network module and the classification network module. The convolutional network module utilizes a convolutional layer and a max-pooling layer to learn high-order representations of feature. Specifically, the convolutional network computes high-order feature maps C by a 2-dimensional convolutional operation and a ReLU activation function, operationally defined as follows:

$$C_i = ReLU(Conv(X_i, W_i, b_i)) \quad i = 0, 1 \quad (9)$$

$$X_i = \begin{cases} X & \text{if } i = 0 \\ P_0 & \text{if } i = 1 \end{cases} \quad (10)$$

where C_i indicates the output of the i -th convolutional layer. W_i and b_i indicates a weight matrix and a bias of the i -th convolutional layer, respectively. $Conv(\cdot)$ defines a convolutional operation. Subsequently, the operation of a max-pooling layer is defined as follows:

$$P_i = MaxPooling(C_i) \quad i = 0, 1 \quad (11)$$

where P_i indicates the output of the i -th max-pooling layer. $MaxPooling(\cdot)$ defines a max-pooling operation.

In the classification network module, the flattened feature vectors are fed into a two-layer feed-forward neural network with a drop layer, and then a softmax function is adopted to perform the prediction. Formally, the operation of the above-mentioned step is defined as follows:

$$x_n = \text{softmax}(\text{MLP}(\text{MLP}(\text{drop}(\text{flat}(P_1))), W_a, b_a), W_b, b_b)) \quad (12)$$

where x_n refers to the prediction value of the n -th input sequence. $\text{softmax}(\cdot)$ defines the softmax function. $\text{MLP}(\cdot)$ defines a feed-forward neural network. $\text{drop}(\cdot)$ defines a dropout operation. $\text{flat}(\cdot)$ defines a flatten operation. W_a, b_a, W_b and b_b are learnable parameters. The detailed architecture of CNN layer is shown in Table 1. We chose the binary cross-entropy loss function, shown as follows:

$$\text{loss} = -\frac{1}{N} \sum_{n=1}^N [y_n \log x_n + (1 - y_n) \log(1 - x_n)] \quad (13)$$

where x_n is the prediction value and y_n is the label.

Table 1. The architecture of CNN layer.

Layer	Input shape	Output shape	Parameters
Convolution2D	(Batch, 768, 200, 13)	(Batch, 256, 66, 26)	3,068
MaxPooling2D	(Batch, 256, 66, 26)	(Batch, 128, 33, 26)	0
Convolution2D	(Batch, 128, 33, 26)	(Batch, 42, 11, 52)	12,220
MaxPooling2D	(Batch, 42, 11, 52)	(Batch, 21, 5, 52)	0
Flatten	(Batch, 21, 5, 52)	(Batch, 5460)	0
Dropout	(Batch, 5460)	(Batch, 5460)	0
Dense	(Batch, 5460)	(Batch, 64)	349,504
Dense	(Batch, 64)	(Batch, 2)	130

2.3 Two-Stage Identification Framework

To facilitate comparison with other methods, we use the same two-stage identification framework as iEnhancer-2L [16]. The identification framework is able to identify not only whether a DNA sequence is an enhancer or not, but also the strength type. Figure 3 provides a visual flow chart showing the working process of our method. The model architecture is identical in both stages, except that the training dataset is different resulting in different parameters. The first-stage model was trained by the benchmark dataset in the Eq. (1); while the second-stage model was trained by Eq. (2). The input DNA sequence is first identified as an enhancer or not by the first stage model. Subsequently, the identified enhancers are further identified as strong enhancers by the second stage model.

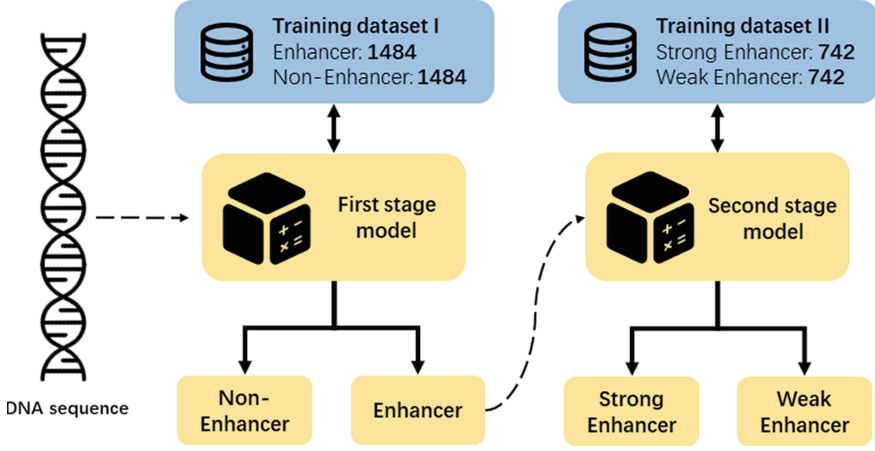


Fig. 3. The flowchart for identifying enhancers and their strength. Training datasets I and II mean E and E^+ of Eqs. (1) and (2). The model architecture for both the first and second stages is the iEnhancer-BERT model architecture.

2.4 Baseline Method

We compare the presented iEnhancer-BERT model with two main categories of enhancer prediction approaches: (i) The methods of predicting enhancers using only sequence features, such as iEnhancer-2L [16], Enhancer-Pred [17], iEnhancer-EL [19], BERT-2D [23] and iEnhancer-XG [20]. (ii) The methods of predicting enhancers using both sequence features and shape features, such as SENIES [21].

2.5 Performance Evaluation Metrics

In this study, we use accuracy, ROC-AUC and MCC to evaluate the performances of iEnhancer-BERT and other baseline methods.

$$Accuracy = \frac{T_+ + T_-}{T_+ + T_- + F_+ + F_-} \quad (14)$$

$$MCC = \frac{(T_+ \times T_-) - (F_+ \times F_-)}{\sqrt{(T_+ + F_+) \times (T_+ + F_-) \times (T_- + F_+) \times (T_- + F_-)}} \quad (15)$$

where T_+ , T_- , F_+ and F_- denote the numbers of true positives, true negatives, false positives and false negative, respectively.

3 Experimental Results

3.1 Different k-mer Pre-training Models

DNABERT [24] provided four pre-training models with different k-mer (3-mer to 6-mer), and we initialized the parameters of the BERT layer using each of these pre-training

models. Subsequently, we evaluated the performance of different k-mer pre-training models on the benchmark dataset, where each sequence was partitioned into several substrings of 3-mer to 6-mer to be fed into the corresponding model. We used three metrics to the overall performance of each k-mer model in five-fold cross-validation. The result of the first stage (enhancer identification) and the second stage (strong enhancer identification) are displayed in Table 2. We found that the performance of the 3-mer pre-training model in the first stage was better than all other pre-training models, while in the second stage, the performance of the 3-mer pre-trained model was about the same as that of the 6-mer pre-training model. This means that the 3-mer pre-training model captures more sequence information in the enhancer identification task, while the 6-mer pre-training model also performs well in the strong enhancer identification task. Combining the performance of the pre-training models with different k-mer in the two tasks, we choose the better-performing 3-mer pre-trained model for the subsequent experiments.

Table 2. Five-fold cross-validation comparison of different k-mer pre-training models on training dataset.

Stages	Model	ACC	AUC	MCC
First stage	3-mer	0.794	0.876	0.593
	4-mer	0.763	0.841	0.533
	5-mer	0.782	0.870	0.566
	6-mer	0.778	0.867	0.560
Second stage	3-mer	0.650	0.703	0.307
	4-mer	0.607	0.683	0.245
	5-mer	0.578	0.679	0.191
	6-mer	0.653	0.697	0.310

3.2 Effect of Pre-training on Model Performance

To better explore the effect of MLM pre-training on model performance, we did not change the structure of our iEnhancer-BERT model and initialized the parameters using the pre-trained model instead of randomly initializing the parameters of the 12-layer Transformer Encoder. We compared the model using pre-training with the model without pre-training on the independent test dataset. As shown in Fig. 4, the performance of both tasks was improved after initializing the parameters of the BERT layer with the pre-trained model provided by DNABERT. In the enhancer identification task (First stage), the AUC of the model with pre-trained parameters was 0.844, an improvement of 0.019 over the model with random parameters. In the strong enhancer identification task (Second stage), the AUC of the model with pre-trained parameters was 0.811, an

improvement of 0.004 over the model with random parameters. The model with pre-training shows more significant improvement for the enhancer identification task, thus illustrating the necessity and effectiveness of the pre-training model.

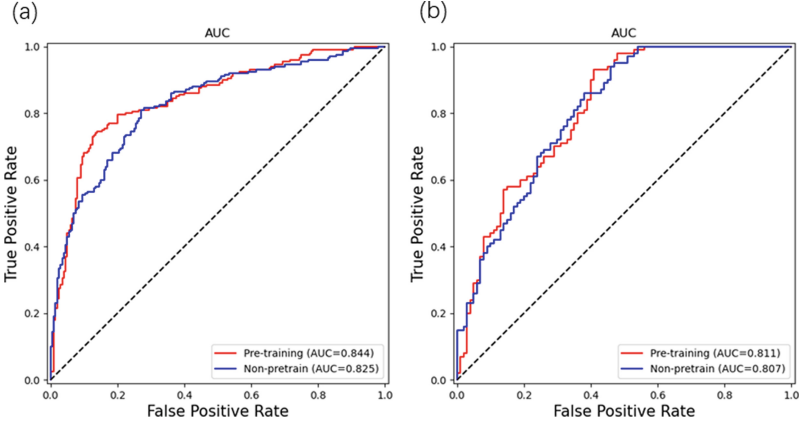


Fig. 4. Performance of AUC comparison the models with and without pre-training on the independent test dataset. (a) First stage. (b) Second stage.

3.3 Effect of Different Fine-Tuning Methods

There are many different fine-tuning approaches based on transfer learning in the BERT architecture. In this section, we compared our model with three other popular fine-tuning methods. These three methods of fine-tuning are shown in Fig. 5, where [CLS] and X_i denote the classification vector and the i -th token representation, respectively, and these methods are described in more detail as follows:

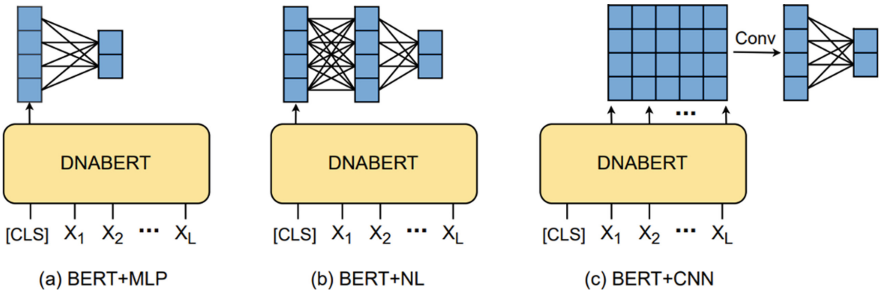


Fig. 5. Three common fine-tuning methods based on DNABERT.

- **BERT + MLP:** In the first fine-tuning method, which is shown in Fig. 5a, nothing changes are applied to the DNABERT. In this model architecture, only the [CLS]

token of the Transformer Encoder last layer output is used as a representation of the whole sequence. The [CLS] token is a vector of size 768 that is fed into a fully connected network with a dropout layer. Finally, it goes through a softmax layer for classification.

- **BERT + NL:** In this method, the classification layer of the original DNABERT is modified. A hidden layer of size 768 is inserted into the fully connected network instead of using a fully connected network without hidden layers, as shown in Fig. 5b.
- **BERT + CNN:** Unlike the previous two approaches that use only [CLS] token as input to the classification layer, in this method in Fig. 5c, all token outputs of the Transformer Encoder last layer are used as input to the convolutional neural networks. After extracting local features by convolution, the flattening vector is fed to the fully connected layer for classification. This method is similar to BERT-2D [23], except that the pre-trained model used in BERT-2D does not contain any prior knowledge of biological sequences.

We evaluate the impact of different fine-tuning methods on the model performance. Figure 6 summarizes the results of our method and the three fine-tuning methods, which are each derived from five-fold cross-validation on the training set to select the best model, and then tested on an independent test dataset. According to Fig. 6, our method achieves optimal results in all performance metrics. In the enhancer identification task (First stage), the ACC, AUC and MCC of our method are 0.793, 0.844 and 0.585, respectively, which are 0.03, 0.003 and 0.06 higher than the next best method (BERT + MLP). In the strong enhancer identification task (Second stage), the ACC, AUC and MCC of our method are 0.701, 0.812 and 0.401, respectively, which are 0.066, 0.069 and 0.127 higher than the next best method (BERT + MLP). In addition, we found that using convolution only for the Transformer Encoder last layer (BERT + CNN) even worse results than the simplest fine-tuning method (BERT + MLP). Intuitively, better results can be achieved by using features from all layers of pre-trained DNABERT fed into the convolutional neural network instead of features from the last layer.

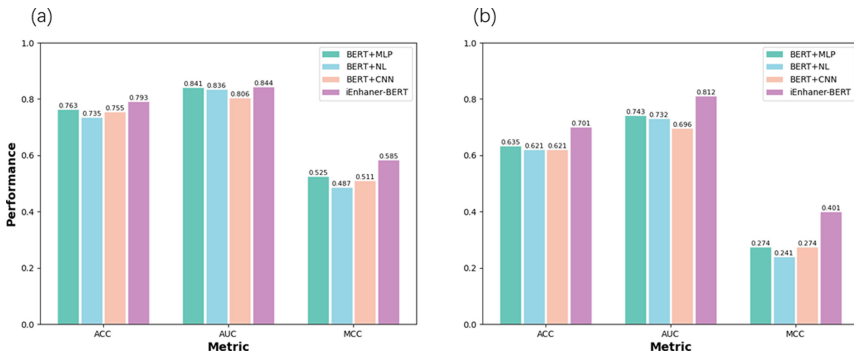


Fig. 6. Performance comparison between our model and three other fine-tuned models on the independent test dataset. (a) First stage. (b) Second stage.

3.4 Performance Comparison with Existing Methods

The identification of enhancers is a focused problem in the field of bioinformatics. Therefore, we compare the performance results with other previously published methods to demonstrate the effectiveness of our method. For a fair and complete assessment of the model performance, comparisons should be made on the same dataset. Note that the source codes for most of the compared methods are not available. Therefore, we directly retrieved the results of other previous enhancer prediction methods (iEnhancer-2L [16], Enhancer-Pred [17], iEnhancer-EL [19], BERT-2D [23], iEnhacner-XG [20] and SENIES [21]). As shown in Table 3, our method achieves the optimal performance on all evaluation metrics in both tasks. For the enhancer identification task, ACC is improved by 0.025, AUC is improved 0.01, and MCC is improved by 0.049. And for the strong enhancer identification task, ACC is improved by 0.018, AUC is improved 0.059, and MCC is improved by 0.021. Overall, it is demonstrated that our method can better extract key features from the sequence information by comparing the performance with existing methods.

Table 3. Performance comparison with existing methods on the independent test dataset.

Stages	Model	ACC	AUC	MCC
First stage	iEnhancer-2L	0.730	0.806	0.460
	Enhancer-Pred	0.740	0.801	0.480
	iEnhancer-EL	0.748	0.817	0.496
	BERT-2D	0.756	–	0.514
	iEnhancer-XG	0.758	–	0.515
	SENIES	0.768	0.834	0.536
	iEnhacner-BERT	0.793	0.844	0.585
Second stage	iEnhancer-2L	0.605	0.668	0.218
	Enhancer-Pred	0.550	0.579	0.102
	iEnhancer-EL	0.610	0.680	0.222
	iEnhancer-XG	0.635	–	0.272
	SENIES	0.683	0.753	0.380
	iEnhancer-BERT	0.701	0.812	0.401

Note: The metrics of other methods were obtained from the corresponding publications.

4 Discussion and Conclusion

In this study, we present iEnhancer-BERT, a novel transfer learning method based on DNABERT for enhancer prediction. Using DNABERT pre-trained model on the whole human genome, we are able to extract deeper feature information of the gene sequence.

iEnhancer-BERT is featured by using all output token information from 12 layers of DNABERT as features and decoding it using CNN layer to complete the classification. Compared to BERT-2D [23] which uses a pre-trained model of natural language as input features, it is more reasonable and robust for us to use a pre-trained model of the human genome. The experiments show that iEnhancer-BERT consistently outperforms several other existing methods on the benchmark datasets. We believe that treating biological sequences as the natural human language for computational modeling is one of the future trends. In conclusion, this work offers the possibility to detect new DNA enhancers.

Although iEnhancer-BERT has achieved good performance in the task of enhancer prediction, there are many research gaps in this problem. Liu et al. [16] split the enhancer sequence into 200 bp fixed-length sequences to facilitate biological modeling. However, the enhancers identified experimentally were variable-length sequences of 50–1500 bp. In addition, experimental studies have shown that enhancers have cell type-specific functions and that different cell types have distinct differences in their enhancers [27, 28]. The existing methods are studied on nine cell types on a generic model, ignoring the cell-specific functions. Therefore, a computational model capable of accurately predicting cell-specific enhancers of variable length will need to be designed in the future.

Due to DNABERT is pre-trained on the whole human genome, our fine-tuning method applies not only to enhancer prediction but also to various DNA sequence classification problems, such as identification of DNA-protein binding sites [29] and DNA N4-methylcytosine sites [30]. Overall, we provide a novel transfer learning architecture to fine-tune DNABERT and contribute to the study of various biological sequences.

Acknowledgments. This work has been supported by the National Natural Science Foundation of China (Grant No. 62002154), Hunan Provincial Natural Science Foundation of China (No. 2019JJ50520, Grant No. 2021JJ40467), Research Foundation of Hunan Educational Committee (Grant No. 20C1579), and Scientific Research Startup Foundation of University of South China (Grant No. 190XQD096).

References

1. Blackwood, E.M., Kadonaga, J.T.: Going the distance: a current view of enhancer action. *Science* **281**(5373), 60–63 (1998)
2. Pennacchio, L.A., et al.: Enhancers: five essential questions. *Nat. Rev. Genet.* **14**(4), 288–295 (2013)
3. Herz, H.M.: Enhancer deregulation in cancer and other diseases. *BioEssays* **38**(10), 1003–1015 (2016)
4. Zhang, G., et al.: DiseaseEnhancer: a resource of human disease-associated enhancer catalog. *Nucleic Acids Res.* **46**(D1), D78–D84 (2018)
5. Mardis, E.R.: ChIP-seq: welcome to the new frontier. *Nat. Methods* **4**(8), 613–614 (2007)
6. Creyghton, M.P., et al.: Histone H3K27ac separates active from poised enhancers and predicts developmental state. *Proc. Natl. Acad. Sci.* **107**(50), 21931–21936 (2010)
7. Heintzman, N.D., et al.: Distinct and predictive chromatin signatures of transcriptional promoters and enhancers in the human genome. *Nat. Genet.* **39**(3), 311–318 (2007)
8. Dorschner, M.O., et al.: High-throughput localization of functional elements by quantitative chromatin profiling. *Nat. Methods* **1**(3), 219–225 (2004)

9. Buenrostro, J.D., et al.: ATAC-seq: a method for assaying chromatin accessibility genome-wide. *Curr. Protoc. Mol. Biol.* **109**(1), 21.29.21–21.29.29 (2015)
10. Firpi, H.A., Ucar, D., Tan, K.: Discover regulatory DNA elements using chromatin signatures and artificial neural network. *Bioinformatics* **26**(13), 1579–1586 (2010)
11. Fernandez, M., Miranda-Saavedra, D.: Genome-wide enhancer prediction from epigenetic signatures using genetic algorithm-optimized support vector machines. *Nucleic Acids Res.* **40**(10), e77–e77 (2012)
12. Rajagopal, N., et al.: RFECS: a random-forest based algorithm for enhancer identification from chromatin state. *PLoS Comput. Biol.* **9**(3), e1002968 (2013)
13. Erwin, G.D., et al.: Integrating diverse datasets improves developmental enhancer prediction. *PLoS Comput. Biol.* **10**(6), e1003677 (2014)
14. Ghandi, M., et al.: gkmSVM: an R package for gapped-kmer SVM. *Bioinformatics* **32**(14), 2205–2207 (2016)
15. Yang, B., et al.: BiRen: predicting enhancers with a deep-learning-based model using the DNA sequence alone. *Bioinformatics* **33**(13), 1930–1936 (2017)
16. Liu, B., et al.: iEnhancer-2L: a two-layer predictor for identifying enhancers and their strength by pseudo k-tuple nucleotide composition. *Bioinformatics* **32**(3), 362–369 (2016)
17. Jia, C., He, W.: EnhancerPred: a predictor for discovering enhancers based on the combination and selection of multiple features. *Sci. Rep.* **6**(1), 1–7 (2016)
18. Liu, B., et al.: iEnhancer-EL: identifying enhancers and their strength with ensemble learning approach. *Bioinformatics* **34**(22), 3835–3842 (2018)
19. Khanal, J., Tayara, H., Chong, K.T.: Identifying enhancers and their strength by the integration of word embedding and convolution neural network. *IEEE Access* **8**, 58369–58376 (2020)
20. Cai, L., et al.: iEnhancer-XG: interpretable sequence-based enhancers and their strength predictor. *Bioinformatics* **37**(8), 1060–1067 (2021)
21. Li, Y., et al.: SENIES: DNA shape enhanced two-layer deep learning predictor for the identification of enhancers and their strength. *IEEE/ACM Trans. Comput. Biol. Bioinform.* (2022)
22. Devlin, J., et al.: BERT: pre-training of deep bidirectional transformers for language understanding. *arXiv preprint [arXiv:1810.04805](https://arxiv.org/abs/1810.04805)* (2018)
23. Le, N.Q.K., et al.: A transformer architecture based on BERT and 2D convolutional neural network to identify DNA enhancers from sequence information. *Brief. Bioinf.* **22**(5), bbab005 (2021)
24. Ji, Y., et al.: DNABERT: pre-trained Bidirectional Encoder Representations from Transformers model for DNA-language in genome. *Bioinformatics* **37**(15), 2112–2120 (2021)
25. Ernst, J., et al.: Mapping and analysis of chromatin state dynamics in nine human cell types. *Nature* **473**(7345), 43–49 (2011)
26. Fu, L., et al.: CD-HIT: accelerated for clustering the next-generation sequencing data. *Bioinformatics* **28**(23), 3150–3152 (2012)
27. Heinz, S., et al.: The selection and function of cell type-specific enhancers. *Nat. Rev. Mol. Cell Biol.* **16**(3), 144–154 (2015)
28. Cai, W., et al.: Enhancer dependence of cell-type-specific gene expression increases with developmental age. *Proc. Natl. Acad. Sci.* **117**(35), 21450–21458 (2020)
29. Shen, L.-C., et al.: SAResNet: self-attention residual network for predicting DNA-protein binding. *Brief. Bioinf.* (2021)
30. Chen, W., et al.: iDNA4mC: identifying DNA N4-methylcytosine sites based on nucleotide chemical properties. *Bioinformatics* **33**(22), 3518–3523 (2017)